

Inter-dataset generalization strength of supervised machine learning methods for intrusion detection

Laurens D'hooge^{a,*}, Tim Wauters, Bruno Volckaert, Filip De Turck^a

^a*Ghent University - imec, IDLab, Department of Information Technology, Technologiepark-Zwijnaarde 126, Gent, Belgium*

Abstract

This article describes an experimental investigation into the inter-dataset generalization of supervised machine learning methods, trained to distinguish between benign and several classes of malicious network flows. The first part details the process and results of establishing reference classification scores on CIC-IDS2017 and CSE-CIC-IDS2018, two modern, labeled data sets for testing intrusion detection systems. The data sets are divided into several days each pertaining to different attack classes (DoS, DDoS, infiltration, botnet, etc.). A pipeline has been created that includes twelve supervised learning algorithms from different families. Subsequently to this comparative analysis the DoS / SSL and botnet attack classes, which are represented in both data sets and are well-classified by many algorithms, have been selected to test the inter-dataset generalization strength of the trained models. Exposure of these models to unseen, but related samples without additional training was expected to maintain high classification performance, but this assumption is shown to be erroneous (at least for the tested attack classes). To our knowledge, there is no prior literature that validates the efficacy of supervised ML-based intrusion detection systems outside of the dataset(s) on which they have been trained. Our first results question the implied link that great intra-dataset generalization leads to great inter- or extra-dataset generalization. Further experimentation is required to discover the scope and causes of this deficiency as well as potential solutions.

Keywords: Binary classification, CIC-IDS2017, CSE-CIC-IDS2018, generalization strength, intrusion detection, supervised machine learning.

1. Introduction

Intrusion detection is a cornerstone of cybersecurity and an active field of research since the 1980s. Although the early research focused more on host intrusion detection systems (HIDS), the principal aims of an intrusion detection system (IDS) have not changed. A well-functioning IDS should be able to detect a wide range of intrusions, possibly in real-time, with high discriminating power, improving itself through self-learning, while being modifiable in its design and execution Denning & Neumann (1985). The advent of computer networking and its ever greater adoption, shifted part of the research away from HIDS to network intrusion detection systems (NIDS).

This article details two experiments, starting with the analysis of two modern intrusion detection datasets CIC-IDS2017 and CSE-CIC-IDS2018 by means of supervised machine learning. The second part is an investigation into the generalizing capabilities of these supervised learners by exposing pre-trained models to unseen attack data from the other dataset. The first part is self-evidently useful, because it is a necessary prerequisite to start the second part. That second part is more innovative and focuses on a research problem that has largely been ignored by intrusion detection researchers. Much effort and resources are spent on improving classification results within data sets (mostly through algorithmic tweaks, ensembles or data prepro-

cessing techniques). Although this is an essential part, those articles conclude with the implicit assumption that an increase in classification performance within the data set, will transfer into a better IDS in real-world scenarios. Omitting to investigate how well the new algorithms perform on unseen attack data, is very likely due to the lack of same-feature, labeled, publicly available data sets. The sparse landscape of good data sets has fairly recently been criticized in Małowidzki et al. (2015) and Creech & Hu (2013). Those publications predate the CICIDS collection which for the first time offers researchers the possibility to test trained models on new, albeit similar, data. Hopefully, as the IDS data generation matures further, from one-time efforts to dynamic generation, generalization testing should become standard practice.

The article is structured as follows. First an overview of the related work in intrusion detection and network security dataset generation is given, then the implementation of the analysis is described (section 3). The fourth part contains the discussion of the results obtained on both datasets with recommendations for algorithms based on raw classification performance and run time characteristics (section 4). The fifth section 5 details the results of the second experiment that looks into generalization strength of the tested algorithms.

Key findings in this work are the outstanding performance both in terms of classification and time metrics of tree-based classifiers, especially ensemble learners and the surprising effectiveness of simple distance-based methods on both datasets.

*Corresponding author: Laurens.Dhooge@ugent.be

These classification results are however not likely to be good proxies for real-world performance of supervised learners of the tested families in intrusion detection. This claim can be made because the methods fail to generalize correctly even under the most favorable conditions.

2. Related work

2.1. Intrusion Detection

The field of network intrusion detection developed two main approaches to solve the problem of determining whether observed traffic is legitimate. The chronologically first approach is the use of signature-based systems (also called misuse detection systems). Within this category different strategies have been studied Axelsson (2000), including state modelling, string matching, simple rule-based systems and expert systems (emulating human expert knowledge, by applying an inference system on a knowledge base). None of the systems in this category, although great at detecting known signatures, generalize. That is a violation of the principles for intrusion detection systems, namely the system's ability to improve itself through learning.

The second approach, anomaly detection, has existed almost as long, but the methods have changed drastically in the past years. Early systems based their decisions on rules, profiles and heuristics, often derived from relatively simple statistical methods. These systems could be self-learning in the sense that their heuristics could be recomputed and could thus become a dynamic part of the system. Advances in the last decade in terms of distributed computation and storage have enabled more advanced statistical methods to become feasible. Work by Buczak et al. Buczak & Guven (2016) concluded that making global recommendations is impossible and the nature of the data and types of attacks to be classified should be taken into account when designing an IDS. Furthermore they emphasize the necessity of training data in the field of network intrusion detection and an evaluation approach that considers more than just accuracy. On a final note, the authors include recommendations for machine learning (ML) algorithms for anomaly detection (density-based clustering methods and one-class SVMs), and for misuse detection (decision trees, association rule mining, and Bayesian networks).

Folino & Sabatino (2016) surveyed the use of ensemble learners including published methods that span the decade between 2006 and 2016. The authors classify surveyed methods into four categories: supervised, unsupervised, hybrid, and other statistical approaches (mainly hidden Markov models). Recommendations for individual approaches are not included in the article. Instead, the authors stress the importance of both distributed- and collaborative- intrusion detection systems. They note that these requirements are mentioned in articles concerning novel singular techniques, but subsequently play no part in further inquiry. The lack of a common testbed to test across multiple data sets and /or testing in more realistic scenarios is criticized by the authors, because the current research method in the intrusion detection community makes comparison (and associated ranking) of results very hard. Some ensemble methods that deserve to be mentioned, but which are not

included in Folino et al. due to their recency are Mirsky et al. (2018), Moustafa et al. (2018), Vaca & Niyaz (2018), Das et al. (2019), Gao et al. (2019) and Das et al. (2020).

Recent work by Hodo et al. Hodo et al. (2017) examines the application of shallow and deep neural networks for intrusion detection. Their graphical overview of IDS techniques clearly shows the dominant position of anomaly-based methods, driven by the adoption of machine learning techniques. Their main contribution is a chapter explaining the algorithm classes of neural networks. The work differentiates between artificial neural networks (ANN) (shallow) and deep networks (DN), with subdivisions between supervised and unsupervised methods for ANNs and generative versus discriminative methods for DNs. Their conclusion is that deep networks show a significant advantage for DNs in detection. They note that the adoption of either class is still in its early stages, when applied to network intrusion detection. In the two to three years after their publication, application of deep neural architectures for intrusion detection has taken off (Yin et al. (2017), Hao et al. (2019), Papamartzivanos et al. (2019), Wang et al. (2019), Liu et al. (2020)).

2.2. Datasets

Self-learning systems require data to train and test their efficacy. All techniques used in this work are supervised, machine-learning algorithms. This means that they do not only require data, but that data has to be labeled. The dataset landscape in intrusion detection has been described among others by Wu et al. Wu & Banzhaf (2010) as part of a review article on the state of computational intelligence in intrusion detection systems and more succinctly by Shiravi et al. Shiravi et al. (2012), as prelude to their efforts in generating a new approach for dataset creation.

This work will only offer a very brief overview of the most frequently studied datasets. KDDCUP99 (KDD99), the subject of ACM's yearly competition on Data mining and Knowledge Discovery in 1999 is by far the most widely studied data set for intrusion detection. It originated in a DARPA funded project, run by the Lincoln Lab at MIT, with the aim of evaluating the state-of-the-art-IDSs at the time. Apart from being based on twenty-year-old data by now, it has also been criticized by McHugh in 2000 McHugh (2000), by Brown et al. in 2009 Brown et al. (2009) and by Tavallae et al. in 2009 Tavallae et al. (2009).

The persistence of a single dataset for almost two decades and its improved version, which now is nearly a decade old, called for new research into dataset generation. The Canadian Institute for Cybersecurity (CIC), a coalition of academia, government and the public sector, based at the University of New Brunswick is the front runner in this field of research. The analysis by Tavallae et al. of the dataset resulted in a new dataset, named NSL-KDD, in which structural deficiencies of KDD99 were addressed. NSL-KDD does not have redundant records in the training data. Moreover, the researchers removed duplicates from testing data and reduced the total number of records so that the entire dataset could be used, instead of needing to

sample it. Finally, to improve variability in the tested learners’ classification ability, items which were hard to classify (a minority of the learners classified them properly) were added to NSL-KDD with a much higher frequency than items which most of the classifiers identified correctly. Because NSL-KDD is a derivation of KDD99, it is not completely free from its origin’s issues.

2.2.1. ISCXIDS2012, CIC-IDS2017 & CSE-CIC-IDS2018

After publishing NSL-KDD, the CIC started a new project to create modern, realistic datasets in a scalable way. The first results from this project are documented in Shiravi et al. (2012). Their system uses alfa and beta profiles. Alfa profiles are abstracted versions of multi-stage attacks, which would ideally be executed fully automatically, but human execution remains an option. Beta profiles are per protocol abstractions ranging from statistical distributions to custom user-simulating algorithms. Building on this foundation, published in 2012, a new dataset was published in 2017 and then another one in 2018. The main difference is that CIC-IDS2017 and CSE-CIC-IDS2018 Sharafaldin et al. (2018) are geared more towards machine learning, with their 80 flow-based features whereas ISCXIDS2012 has 20 packet features. All three datasets give access to the raw pcap files, for further analysis. The flow features were gathered with CICFlowMeter, an open source flow generator and analyzer. CIC-IDS2017 added an HTTPS beta profile, which was necessary to keep up with the surge in HTTPS adoption on the web (Google transparency report).

3. Architecture and Implementation

The evaluation of this dataset is a project in Python, supported by the Pandas McKinney (2010), Sklearn Pedregosa et al. (2011) and XGboost Chen & Guestrin (2016) modules. The following subsections detail the engineering effort and choices to produce a robust, portable solution to evaluate any dataset. Google’s guide Zinkevich, Rules of Machine Learning: Best Practices for ML Engineering, by Martin Zinkevich, has been influential on the implementation (mainly rules 2, 4, 24, 25, 32 and 40), as well as the detailed guides offered by Scikit-Learn. The current implementation makes use of twelve supervised machine learning classifiers. Seven tree-based algorithms: a single decision tree (CART) (dtree), random forests (rforest), a bagging ensemble learner built on decision trees (bag), gradient-boosted trees (gradboost), random decision trees (extratree), adaboost (ada) and extreme gradient-boosted trees (xgboost). Two neighbor-based ones: the K-nearest neighbor classifier (knn) and the N-centroid classifier (ncentroid), two SVM-based methods: linearSVC (liblinear subsystem) (linsvc) and RBFSVC (libsvm subsystem) (rbfsvc) and one logistic regression (L-BFGS solver) (binlr).

3.1. Data Loading and Preprocessing

The datasets consist of labeled flows for eight and ten days respectively. A merged version of CIC-IDS2017 has also been created. Details about the included attack types, dataset sizes

and sample distributions are listed in tables 1 and 2. Each day has the same features, 84 in total (label not included). It should be noted that the ”Fwd Header Length” feature is duplicated in CIC-IDS2017, an issue of CICFlowMeter that has been fixed in the source code, but persisted in the dataset. Another caveat when importing this data for analysis, is the presence of the literal value Infinity. String-type data like this results in run time crashes, when mixed with numeric data. This was rectified by replacing the strings with NaN values. The Label column was binarized. While this does incur information loss, it is justified for an outer defense layer to classify first between benign and malign traffic classes instead of individual attacks. This squashing only affected the DoS-SSL, brute force and web attack subsets of CIC-IDS2017 as the other subsets already had only one label for the malicious traffic. For CSE-CIC-IDS2018 the same classes are affected with the addition of DDoS. Almost all cases are a reduction of 2 attacks from the same attack type under 1 label.

Table 1: CIC-IDS2017 attack type, size, baseline count, attack count.

Dataset files			
Attack type(s)	Size (MB)	Baseline	Attack
No attacks	231	529918	0
FTP / SSH bruteforce	173	432074	13835
Layer 7 DoS and Heartbleed	283	440031	252672
Web attacks	67	168186	2180
Infiltration	108	288566	36
Ares botnet	75	189067	1966
Nmap port scanning	101	97718	128027
Layer 4 DDoS	95	127537	158930
Merged	1100	2273097	557646

Table 2: CSE-CIC-IDS2018 attack type, size, baseline count, attack count.

Dataset files			
Attack type(s)	Size (MB)	Baseline	Attack
FTP-SSH brute force	358	667626	380949
HTTP-DoS	376	996077	52498
HTTP-DoS	334	601802	446772
DDoS	384	576191	472384
DDoS	329	687742	360833
Web attacks	383	1048213	362
Web attacks	383	1048009	566
Infiltration	209	544200	68871
Infiltration	108	238037	93063
Botnet	352	762384	286191

3.2. Cross-validation and Parameter Optimization

The implementation has two main branches, cross-validated parameter optimization and single execution testing. The branches share all code up to the point where the choice of algorithm

is done. For parameter tuning, K-fold cross-validation is employed, with $k = 5$. The splits are stratified, taking samples proportional to their representation in the class distribution.

Parameter tuning is done with grid search, evaluating all combinations of a parameter grid. This is multiplicative: e.g. for two parameters, respectively with three and five values, fifteen combinations are tested. An overview of the algorithms and their parameter search spaces can be seen in table 3.

Special care has gone into avoiding model contamination. The data given for cross-validated parameter tuning is two thirds of that day's data. Optimal parameters are derived from only that data. The results from cross-validation are stored. These results include: the total search time, the optimal parameters, the parameter search space and the means of five metrics (balanced accuracy, precision, recall, f1-score and ROC-AUC).

Table 3: CIC-IDS2017, CSE-CIC-IDS2018 algorithm, parameters, ranges.

Parameter tuning search space		
Algorithm	Parameters	Search space
dtree	max_features	2 .. 80
	max_depth	1 .. 35
bag	max_features	0.1 .. 1.0
	max_samples	0.1 .. 1.0
rforest	max_features	2..80
	max_depth	1 .. 35
extratree	n_estimators	10 .. 100
ada	n_estimators	10 .. 100
	learning_rate	0.1 .. 1.0
gradboost	n_estimators	10 .. 100
	learning_rate	0.1 .. 1.0
xgboost	n_estimators	10 .. 100
	learning_rate	0.1 .. 1.0
knn	n_neighbors	1 .. 5
	distance_metric	manhattan euclid
linsvc	max_iterations	$10e3 \cdot 10e6$
	tolerance	$10e-3 \cdot 10e-5$
binlr	max_iterations	$10e3 \cdot 10e6$
	tolerance	$10e-3 \cdot 10e-5$

3.3. Metric Evaluation and Model Selection

The only point of interaction between the cross-validation (cv) code and the single execution is in gathering the optimal model parameters from the result files, written to disk by the cv code. The optimal parameters are chosen, based on a voting system. That voting system looks at the ranks each set of tested parameters gets on the following five metrics:

Balanced accuracy: combined per class accuracy, useful for skewed class distributions. Obtained through evaluating equation 1 for each class, averaged, compared to accuracy itself which also uses equation 1 over all classes simultaneously. TP, TN, FP, FN respectively stand for true / false positive / negative.

$$ACC = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

Precision: Of the items that were tagged as positive, how many are actually positive (equation 2).

$$PR = \frac{TP}{TP + FP} \quad (2)$$

Recall: Of the items that were tagged as positive, how many did we tag compared to all positive items (equation 3).

$$RC = \frac{TP}{TP + FN} \quad (3)$$

F1-score: defined as the harmonic mean of precision and recall (equation 4), the F1-score combines these metrics in such a way that the impact of poor scores on either of the metrics, heavily impacts the final score. In order to achieve a high F1-score, it is not only sufficient to be precise in prediction (discriminative power), but equally high in finding a generalized representation of positive samples.

$$F1 = \frac{2 * precision * recall}{precision + recall} \quad (4)$$

ROC-AUC: the receiver operator characteristic (ROC) is a visual metric of the relationship between the true positive rate (recall) on the y-axis and the false positive rate (equation 5) on the x-axis at different classifying thresholds. The thresholds are implicit in the curve. In essence it shows how well a classifier is able to separate the classes. To avoid having to interpret the plot, the area under the curve (AUC) is calculated. An AUC of 1 would mean that the classifier is able to completely separate the classes from each other. An AUC of 0.5 indicates that the class distributions overlay each other fully, meaning that the classifier is not better than random guessing. The AUC reduces the ROC curve to a single number. If special care has to be given to the avoidance of false positives or to maximal true positive rate, then the AUC metric is no longer helpful. For unbalanced data sets, the ROC curve is a great tool, because the imbalance is irrelevant to the outcome.

$$FPR = \frac{FP}{FP + TN} \quad (5)$$

The optimal parameters are decided by a voting mechanism that works as follows: 1: Find the highest ranked sets of parameters for each of the five metrics. 2: Aggregate across the found sets. 3: Pick the set that is best-ranked most consistently. Some algorithms showed a high preference for certain parameter values. The results are summarized in table 4. A singular dash indicates that the distribution of chosen parameter values was too uniform. In some cases a clear grouping exists around a subspace of the search. Values were included if at least half of the cross-validated models landed on a small subrange of the total parameter range (counts indicated in parentheses).

3.4. Algorithm Retesting with Optimal Parameters

For each day (each attack scenario), the algorithms were retested with optimal parameters. Seven classification metrics of this execution are gathered, namely the five metrics used to evaluate the hyperparameters in the cross-validation phase (described in paragraph 3.3). In addition the accuracy score is kept as well as the confusion matrix. The used parameters and run time of the algorithm have been kept as well.

Table 4: CIC-IDS2017 / CSE-CIC-IDS2018 optimized parameter values if any for each algorithm

Parameter tuning search results			
Algorithm	Parameters	2017	2018
dtree	max_features	-	-
	max_depth	-	-
rforest	max_features	-	-
	max_depth	-	-
extratree	n_estimators	-	-
bag	max_features	0.8 / 0.9 (12/24)	0.8 / 0.9 / 1.0 (20/30)
	max_samples	0.9 / 1.0 (18/21)	0.8 / 0.9 / 1.0 (22/30)
ada	n_estimators	-	-
	learning_rate	-	0.1 / 0.3 (17/30)
gradboost	n_estimators	100 (12/24)	80/90/100 (19/30)
	learning_rate	0.1 / 0.2 (15/24)	0.1 / 0.2 (15/30)
xgboost	n_estimators	-	90 / 100 (19/30)
	learning_rate	-	-
knn	n_neighbors distance_metric	1 (21/24) manhattan (23/24)	1 (22/30) manhattan (18/30)
linsvc	max_iterations	100 / 10000 (12/16)	-
	tolerance	-	10e-5 10e-6 (12/20)
binlr	max_iterations	100 / 1000 (15/16)	100 / 1000 (16/20)
	tolerance	10e-3 (16/16)	10e-3 (20/20)

4. Evaluation Results

This section describes the results from the retesting with optimized parameters. In total 12 algorithms were tested. It should be noted that for two of these no cross-validation was done. The N-centroid classifier does not use optimized parameters, due to a limitation of Scikit-learn. For the RBF-SVC classifier, parameter optimization was skipped due to the excessive run times of forced single-core execution. The results are described in their respective algorithmic classes in subsections 4.1 and 4.2. All testing was done on a server equipped with 2X Intel Xeon E5-2650v2 @ 2.6GHz (16 cores) and 48GB of RAM.

4.1. CIC-IDS2017 Classification Results and Discussion

This section details the results of testing the various classification algorithms. The algorithms are grouped, based on their underlying classifier. Due to page constraints the complete set of results in tabular format is omitted from the article, but made available publicly at Gitlab. It is advised to use that material

with the full collection of tables and derived graphs. A single graph of the results of one algorithm on one attack class is shown in figure 1.

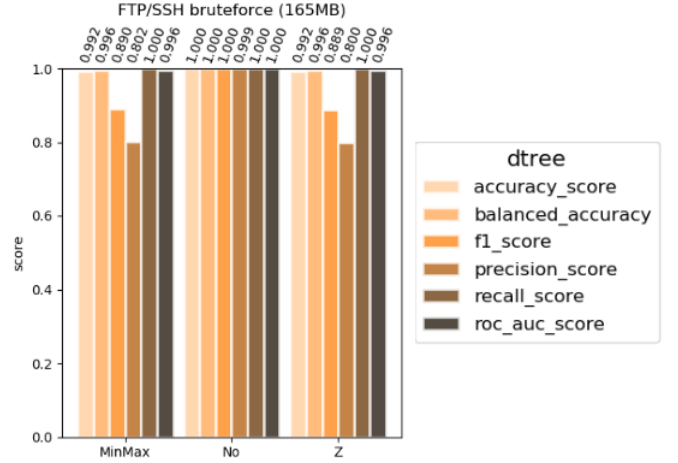


Figure 1: Sample result graph, full results (graphs and tables) available at <https://gitlab.ilabt.imec.be/lpdhooge/CIC-IDS2017-2018-ml-graphics>

4.1.1. Tree-based Classifiers

On the whole, the tree-based classifiers obtain the best results for all attack types, on all metrics. Even a single decision tree is able to achieve 99+% on all metrics for the DoS / DDoS and Botnet attack types. Whether or not feature scaling is applied has mixed outcomes. The difference in performance can be quite severe with up to a 15% absolute difference in metric scores. Results on the merged dataset reveal that identification across different attack classes works with equally great results to the best-identified classes. It should however be noted that good performance on the merged data set includes the attack classes with the most samples (DoS, port scan & DDoS) and thus obfuscates worse performance on the less prevalent attack classes.

When introducing meta-estimators, techniques at a higher level of abstraction that introduce concepts to improve the underlying classifier(s), several benefits were discovered.

The bagging classifier proved itself to be a more potent meta-estimator, reaching near-perfect scores on all metrics for the brute force, Dos, DDoS, botnet and port scanning traffic. The improved performance and stability in the brute force and botnet classes compared to plain decision trees and random forests is its main advantage. Scoring on the infiltration attacks improved, regardless of scaling. This is believed to be a result of the resampling employed by the bagging classifier. The almost perfect classification on five of the seven attack classes generalized to the evaluation of the entire data set.

Random forests improved the results on port scanning and web attack traffic, now abstracting over the scaling choice compared to a decision tree. For the other attack types results are very similar, with a noteworthy reduction on FTP/SSH traffic

classification, but only when using No scaling. An improvement on multiple classification metrics is observed for the infiltration attacks as well. This attack type consistently is the hardest to classify, not least because the dataset only contains 36 of these flows, compared to the 288566 benign samples in the same set (1). The overall stability is the most useful result. Abstraction over the scaling choice is present for all attack classes.

Randomized decision trees (extratree) improve over random forests, reaching perfect classification for brute force traffic. For infiltration traffic the best results yet are achieved reaching couples of perfect precision with 75% recall (no scaling or normalization). Compared to the random forest it does lose a couple of percentage points on all metrics when trying to classify botnet traffic. When also considering the computational efficiency of randomized decision trees, this method should be considered as a prime candidate for a real intrusion detection system.

Thanks to its focus on improving classification for difficult samples, adaboost is a very strong classifier in terms of absolute performance. Perfect classification is a reality, regardless of scaling, for brute force, DoS, port scan and DDoS traffic. Botnet and web attack traffic recognition has minimal variation, hovering around 99% on all metrics. When no scaling is applied, the recall scores on infiltration traffic still stand at 75%, but the precision lowers to maximum 75% as well.

Gradient-boosted trees follow a different boosting paradigm than adaboost, but yield very comparable results. Small losses in classification performance have been observed for botnet and web attack traffic. Classification of infiltration traffic is worse by a substantial margin with more variability with regard to pre-processing compared to adaboost.

Extreme gradient-boosted trees (XGBoost) ties with adaboost to be chosen as the best classifier in terms of absolute metric performance. Compared to adaboost it fares better on infiltration traffic because it has better precision and recall scores regardless of scaling. The only disadvantage is a reduction in recognition of brute force traffic. That reduction is also dependent on the scaling method.

4.1.2. SVM-based Classifiers and Logistic Regression

This subsection covers three more algorithms, two support vector machines: one with a linear kernel and one with a radial basis function kernel and a logistic regression classifier.

The linear support vector machine consistently has very high scores on all metrics for the DoS, DDoS and port scan attack classes. Recall on the ftp/ssh brute force, web attacks and botnet attack classes is equally high, but gets offset by lower precision scores. The algorithm thus succeeds in recognizing most of the attacks, but has higher false-positive rates compared to the tree-based methods. Precision on the infiltration and attack classes is extremely poor. The logistic regression with binomial output results is very similar to the linear support vector machine. Like the linear support vector classifier (linSVC), it performs best and most stably on the DoS, DDoS and port scanning traffic. Furthermore, recall scores on the brute force, web and botnet traffic classes are high to very high, but paired with

worse precision scores compared to the linSVC, the applicability of this model gets reduced. Performance is reasonable on the merged data set with standardized features (96.8% recall & 71.8% precision), but not even close to the performance of the tree-based classifiers. This performance reduction does indicate that misclassification on the less prevalent classes is impactful on the final scores. The last algorithm in this category, a support vector classifier with radial basis function kernel shows no surprise results. Classification results are great for the classes on which other methods did well (DoS, port scan & DDoS), high recall and low precision for the botnet, brute force and web attacks and dismal performance on infiltration traffic.

4.1.3. Neighbor-based Classifiers

Despite its simplicity, the k-nearest neighbors algorithm, generally looking at only one neighbor and using the Manhattan (block) distance metric, is a high-performer in 6/7 attack scenarios. In four scenarios, 99.9% on all metrics is almost invariably obtained, with metrics for the other attack classes generally in the mid- to high nineties. The elusive class to recognize remains infiltration traffic. Interestingly enough, even though this too is a distance-based algorithm, all feature scaling methods, yielded very similar results. Another conclusion is the loss of perfect classification, compared to the tree-based classifiers. While the reduction in classification performance is minute, it is observable and stable. Performance on the merged data set shows generalization capability, but this comes at a cost further described in subsection 4.3. The last algorithm, nearest centroid classifier is equally simple and has some interesting properties. Its results resemble the SVM results, with high, stable recognition of DDoS and DoS traffic, mediocre but stable results in the port scanning category and medium to high recall on the web attack, botnet and infiltration traffic, but paired with low precision scores. Combined with its run time profile, it has application potential for the recognition of DDoS and port scanning traffic. On the merged data set results show degraded performance, reflective of the poor classification scores on the other attack classes.

4.1.4. Discussion

The classification results on CIC-IDS2017 are extremely high for almost all methods even though obvious contaminant features (e.g. source / destination IP) had been removed prior to training and evaluation. Only aggressive data reduction, both in terms of training volumes and feature access did lower classification scores. Still, for classes such as DDoS, DoS-SSL and botnet, with only access to a couple hundred training samples and the loss of the 20 most discriminative features to build models, performance remained at the levels presented in this text. Either CIC-IDS2017 is easy to classify through its abundance of samples and features or the methods overfit so fast that even harsher constraints should be put in place to prevent this. Further analysis is required to be more conclusive about this finding.

4.2. CSE-CIC-IDS2018 Classification Results and Discussion

4.2.1. Tree-based Classifiers

Regardless of scaling, when exposed to 66% of the data to train on, a single decision tree reaches perfect classification according to all metrics for the brute force, DoS, DDoS and botnet traffic classes. Results on the two days containing web attack traffic are still good, with metrics stably well above 90%. The two days of infiltration traffic show a different result, with one day so poorly categorized that the ROC-AUC scores report near-perfect overlap between the benign and malign traffic. The second day of infiltration traffic is well-recognized in terms of recall (99+%), but with precision around 79%.

The bagging classifier with underlying decision trees maintains the perfect classification on the 4 classes for which its singular part reaches the same outcomes. The advantages of the bagging classifier include a marginal increase in recognition of the web attack traffic and almost a 20% increase in the precision on the second day of infiltration traffic, now closely near the recall around 92-93%. The results show that the meta-estimator once more abstracts over the scaling choice.

Closely related to the bagging classifier, but with a degree of randomization are random forests. Unsurprisingly they behave very similarly. The marginal increase in performance over a stock decision tree when classifying web attacks is maintained, but the increase in precision on the second day of infiltration traffic is lost if minmax or no scaling was used during preprocessing. Randomized decision trees (extratree) perfectly classify the easy classes, but show a stable, distinct drop in metric scores on the second day with web attack traffic and the second day with infiltration traffic. The computational efficiency of this classifier should give it an edge when deciding on a method to classify the easy classes in a real system.

Despite its reweighting strategy to classify difficult samples, adaboost proves no more potent than the previous estimators on the first day of infiltration traffic. It strands roughly where the other estimators got in terms of classification performance. This is odd because of the 65000+ samples that were available now, compared to the 36 in CIC-IDS2017. It might be possible that this class is fundamentally different from the others. The comparatively great performance on the second day of infiltration traffic can be attributed to the inclusion of port scanning traffic in the infiltration class as opposed to being a separate category. Adaboost kept the perfect scores on the easy classes and performed about as well as the other meta-estimators on web traffic.

As was the case with performance on CIC-IDS2017, gradient-boosted trees show increased variability in metric scores, most often in a downward direction. On CSE-CIC-IDS2018 these fluctuations are most visible in the recognition of web attack traffic with swings of 40% or more. The instability of this method means that it should be used more cautiously.

Extreme gradient boosted trees (XGBoost) dethrone adaboost as the best tree-based classifier. This is mainly due to the score improvements for web attack traffic. Performance on the infiltration class is practically indistinguishable from adaboost's. The numbers show perfect classification regardless of scaling

for brute force, DoS, DDoS and botnet traffic. With minmax or no scaling the first day of web attack traffic is perfectly classified as well. The second day has balanced accuracy scores around 95-96%.

4.2.2. SVM-based Classifiers and Logistic Regression

A support vector machine with a linear kernel also manages to classify the four easy classes, indicating that these are distinctly different from baseline traffic. Although there is some variability at the upper end compared to the perfect scores of the tree-based methods, this classifier should not be discarded because it is computationally very fast after training. The final interesting finding of the linear SVC is the near perfect recall on web attack traffic.

The logistic regression is a mirror-image of the linear support vector machine. It has (near) perfect scores on the easy classes and similarly high recall and low precision on web attack traffic. The performance on both days with infiltration traffic is nowhere close to being useful for real intrusion detection systems.

Changing the kernel of the SVM to a radial basis function has little effect. The easy classes remain very well recognized. Very high recall paired with very low precision is still the result for web attack traffic. What's most interesting about this classifier is the recall score when using normalized features on the first day of infiltration traffic. It stands at 94.9%, higher than any other classifier. The associated precision is only 23% indicating that the method is very proficient at filtering the attack traffic from the baseline, but then relatively poor at placing the samples in their proper class.

4.2.3. Neighbor-based Classifiers

K-nearest neighbors should benefit the most from having access to 66% of the data to train on. This training is no more than placing these samples in a data structure like a ball-tree that is better suited to neighbor search than a plain table. The abundance of samples for comparison leads to recognition of the easy classes with perfect scores. How the features are scaled is not important to this method. Classification of web attack traffic is very good, but the tree-based learners do hold an edge. The second day of infiltration traffic is moderately well-classified.

The simplest method, the nearest centroid classifier remains a good choice to extract DoS and DDoS traffic. If the precision is lacking, the method often has good recall. The results however are not stable with regard to the scaling choice for the features. When comparing its results on CIC-IDS2017 this limitation becomes more clear. The best example is the perfect recall of botnet traffic only obtained after standardization on CIC-IDS2017 and no scaling on CSE-CIC-IDS2018.

4.2.4. Discussion

The discussion for CIC-IDS2017 (4.1.4), applies to CSE-CIC-IDS2018 as well. Only aggressive reduction in data access lowers the classification performance and attack types such as DDoS, DoS-SSL and botnet are least affected by this. Even infiltration attacks are classified well even though the nature of

it did not change, supporting the hypothesis that the oversupply of data is not beneficial for classification, but rather leads to the existence of patterns representative of the dataset, but not of the attack itself. There is ample room to build optimized models, trained with far less data. This would certainly improve the computational requirements and potentially improve the inter-dataset generalization.

4.3. Time Performance Comparison

Real-world intrusion detection systems have constraints, when evaluating traffic. Some example constraints are a.o.: required throughput, minimization of false positives, maximization of true positives, evaluation in under x units of time, real-time detection, etc. To gain insight in the run time requirements of the different algorithms, summarizing charts can be seen in figures 2, 3, 4 and 5. Several main takeaways should be noted from these charts. First, most single executions stay under a minute run time. This time includes model training, but not reading and preprocessing of the data. Methods that deviate almost invariably from this rule of thumb include k -nearest neighbors and the RBF-kernel SVM. This limitation is inherent to the algorithm design of k -nearest neighbors. The RBF-kernel SVM is held back by the implementation in libsvm, locking execution to a single core. A 350 MB data set containing the 80 features typically holds around a million flows. Second, the tree-based learners have the lowest run times, but adaboost and gradient-boosted trees should be more fine tuned during training, because they sometimes fail to converge, before hitting the maximum number of boosting rounds. Third, the nearest centroid classifier is as good as insensitive to data set size. In combination with the classifier's ability in recognizing DDoS and port scanning traffic, it is conceivable to employ it in real-time on IoT networks to identify compromised devices, taken over to execute DDoS attacks (Cloudflare-blog). Fourth, the merged version of CIC-IDS2017 is a 1.1 GB data set with more than 2.8 million flows yet evaluation from start to finish is done within a couple of minutes for most of the algorithms. Fifth and finally, the overall recommendations based on the run times of the individual algorithms should be to favor randomized decision trees and extreme gradient boosted trees. These methods are top performers on the attack classes included in both datasets, both in terms of classification scores and run times.

4.4. Result Comparison to State of the Art

Because of the recency of CIC-IDS2017 and -2018, published research is still limited. Nonetheless a comparison to a selection of relevant research is already possible.

Attak et al. Attak et al. focus on the DARE (data analytics and remediation engine) component of the SHIELD platform, a cybersecurity solution for use in software defined networks (SDN) with network function virtualization (NFV). The machine learning methods that were tested are segmented into two classes: those for anomaly detection and those for threat classification. Comparison to this work is apt for the threat classification portion. The researchers kept only a 10-feature subset of the flow data. This subset was chosen, not produced

by a feature selection technique. The threat classification made use of the random forest and multi-layer perceptron classifiers. Optimal models from a 10-fold cross-validation were used on 20% of the data that was held out for validation. The random forest classifier obtained the best results on accuracy, precision and recall, often reaching perfect classification. The multi-layer perceptron had similar accuracy scores, but much greater variability on precision and recall. Overall their research shows favorable results for the random forest, both in terms of performance on the tested classification metrics, but also on execution time.

Marir et al. Marir et al. (2018) propose a system for intrusion detection with deep learning for feature extraction followed by an ensemble of SVMs for classification, built on top of Spark, the distributed in-memory computation engine. The feature extraction is done by a deep belief network (DBN), a stack of restricted Boltzmann machines. Next, the dimension-reduced sample set is fed to a layer of linear SVMs for classification. Layers further in the stack of SVMs are given samples for which previous layers were not confident enough. Because both the DBN and the SVMs operate in distributed fashion, the master node decides whether enough data is present to build a new layer. If not, then the ensemble of SVMs are the models produced in the final layer. This approach was tested on four datasets, namely KDD99, NSL-KDD, UNSW-NB15 and CIC-IDS2017. The results of testing their approach reveal that the combination of deep learning for feature extraction and supervised learning with an emphasis on retraining for difficult samples, yields better performance on the classification metrics, but requires more training time than the application of the individual parts on the classification task.

5. Model Generalization Strength

As a practical defensive tool, intrusion detection should ideally be tested in the real world. Experiments that push the boundaries in this regard are very rare (Probst T. & V. (2015), Viegas et al. (2017)). Equally uncommon is the adoption of machine learning based solutions into commercial IDSs. This split between the research community that studies intrusion detection through machine learning methods and the real-world adoption of systems built on the research findings has been described most succinctly in Sommer & Paxson (2010). The authors remark that the borrowing of machine learning methods to apply to a sparse landscape of good data sets is an insufficient strategy to make actual progress in the field. They recommend a better understanding of the threat model before designing an IDS and keeping the scope of the resulting system narrow while reducing the (human) cost of interpreting the alerts. The shortcomings of anomaly detection as an approach to network security might be inherent to the data on which they are supposed to operate and the assumptions that were gratuitously taken from Denning & Neumann (1985) might not have held or no longer hold (Gates & Taylor (2007)). The introduction of the new datasets might have a distracting effect, once more pushing researchers to develop novel combinations of or tweaks to algorithms which then achieve (marginally) better metric scores

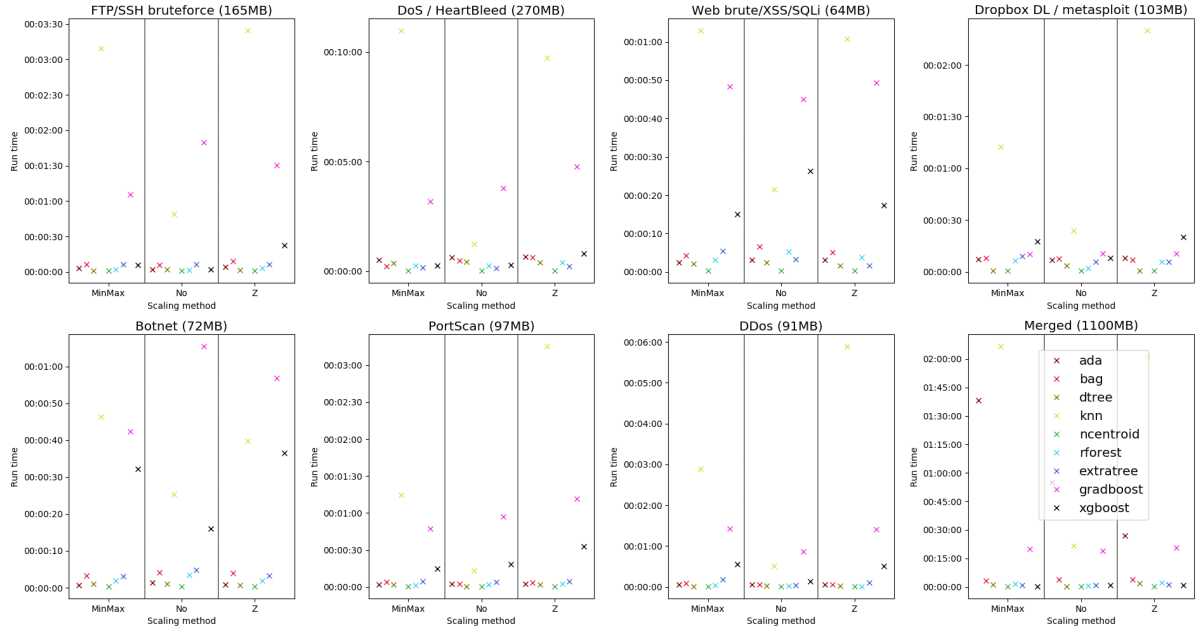


Figure 2: CIC-IDS2017: algorithm run times scatter part 1

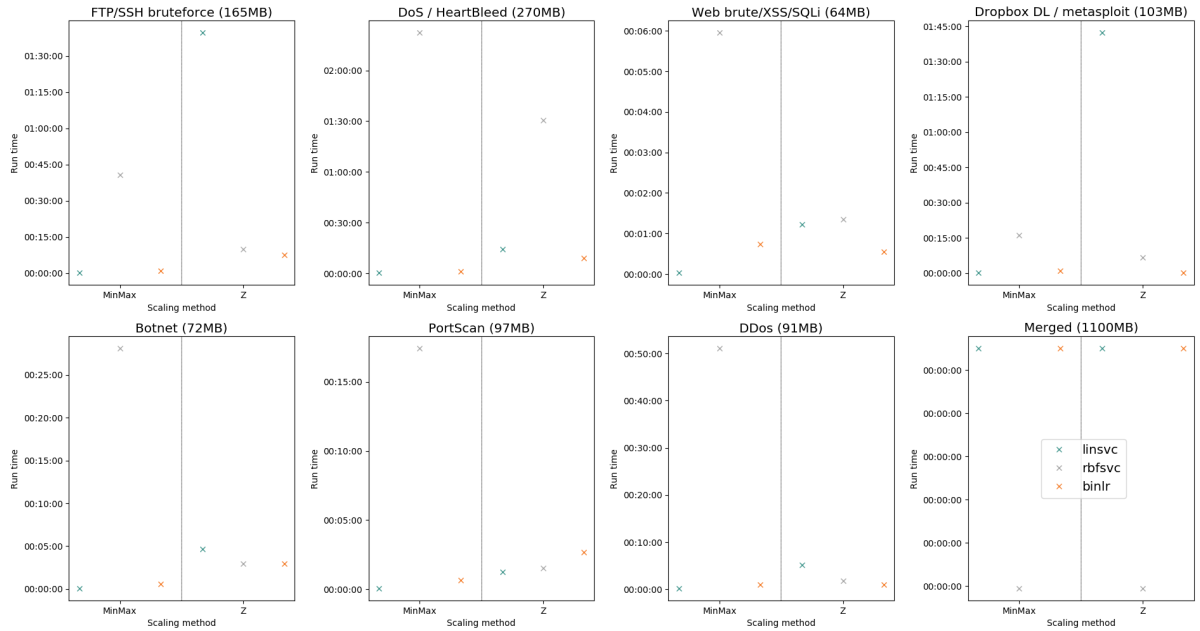


Figure 3: CIC-IDS2017: algorithm run times scatter part 2

than the state of the art. This article aims to get ahead of the problem by showing that the assumption of generalization of models trained on this new data is fictitious. More research is needed, as the experiment described here only tests one attack class that is perfectly classified by the algorithms in both data sets. The uncovered deficiencies might not exist for the other attack classes or might not be insurmountable. In the event that they are, alternative approaches should be considered. These can include building intrusion detection tailored to a specific attack class Robertson et al. (2006), a specific protocol Helle-

mons et al. (2012) or even a specific protocol attack class Jazi et al. (2017).

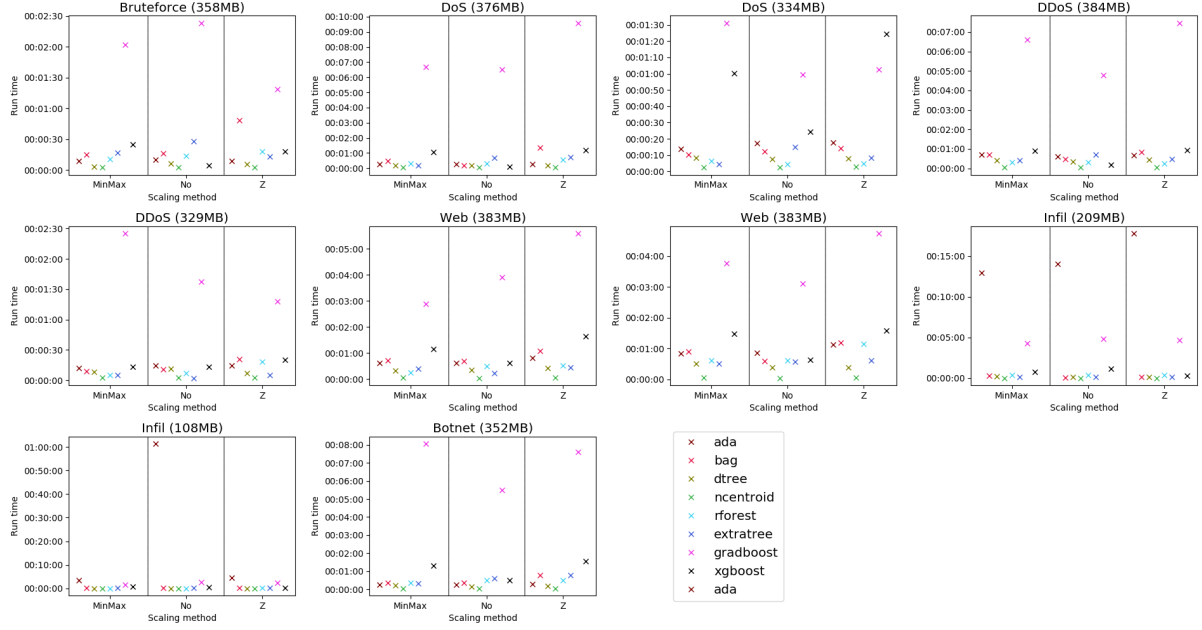


Figure 4: CSE-CIC-IDS2018: algorithm run times scatter part 1

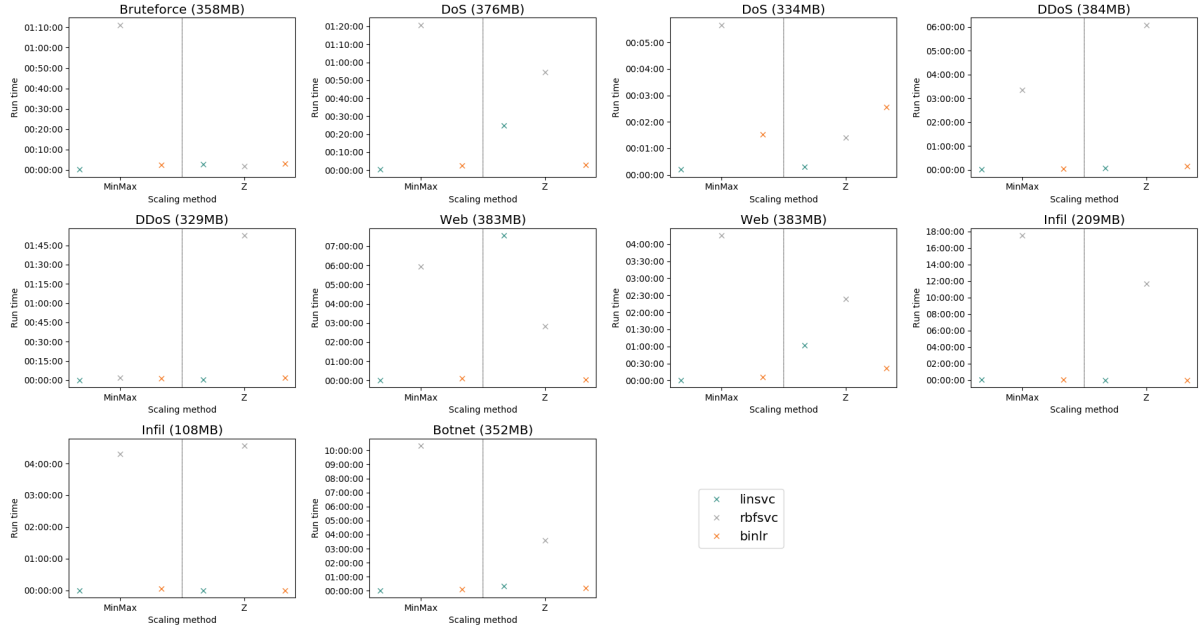


Figure 5: CSE-CIC-IDS2018: algorithm run times scatter part 2

5.1. Generalization Experiment Design

Sections 4.1 and 4.2 show that (near-) perfect classification on most attack classes in the two data sets is possible. Combined with the run time profiles of these methods, they would be recommended for inclusion in real intrusion detection systems. This section details the efforts to test if these methods stay so performant when exposed to unseen, but related data. The conditions to test generalization strength of the trained models are optimal. CIC-IDS2017 and CSE-CIC-IDS2018 originate from the same experimental setup. The biggest difference between

the two versions is that the 2018 version is hosted in the cloud with a different network layout and larger subnetworks (resulting in larger data captures). The attacks have also been split over more days than in the 2017 version, and the tools that were used to attack have only expanded slightly. Because of the very high similarity, it is expected that the trained models would be capable of maintaining their classifying performance. The DoS-SSL and botnet classes have been chosen as first candidates to test this hypothesis. CIC-IDS2017 has four layer 7 DoS attacks and an active exploit of the heartbleed vulnerabil-

ity in the Wednesday data (labeled as day 1 in the analysis). CSE-CIC-IDS2018 uses the same tools, but split the Dos attacks and the heartbleed exploit into two separate days (labeled as 1 and 2 in this analysis). Since the former contains the superset of the attacks in the latter two, it would be expected that the models trained on day 1 of the 2017 data perform equally well on day 1 and 2 of the 2018 data. CIC-IDS2017 only has samples of nodes infected with the ARES botnet, while CSE-CIC-IDS2018 adds the ZEUS botnet. If the algorithms were able to incorporate a generalized knowledge of botnet behavior, it is expected that the addition of ZEUS botnet samples will not weaken the classification potential of models trained on the botnet data of CIC-IDS2017.

All figures present the generalized balanced accuracy and F1-score of each of the twelve algorithms and each of the three scaling methods. Scores are bound on the low-end by zero and on the high-end by one. The title of each figure is its data subset to pretrained model pairing. Figure 6 shows the markers for the classification metrics as well as a distinct color to represent each algorithm. The ordering of the methods in the legend matches the ordering of the methods in each figure.



Figure 6: Legend for all subsequent figures related to model generalization performance scores

5.2. Generalization of the DoS-SSL Attack Class

The previous subsections keep the models and the data they were trained on aligned or only mixed within the same data set. This section moves further by introducing the crossover between data sets. Models trained on the 2017 DoS-SSL data are reevaluated with data from the 2018 data set and vice versa.

The models of DoS subset 1 (layer 7 attacks) of CSE-CIC-IDS2018 performed reasonably well on the 2017 DoS data (fig. 7), reaching balanced accuracies around 80%. For tree-based methods recall is the limiting factor hovering around 50% for the successful algorithms. The F1-scores for tree-based classifiers consist of almost precision, but paired with the sub-par recall of attacks ($\sim 50\%$) overall performance is not strong. The SVCs and logistic regression have the inverted conclusion with better F1 scores, due to high recall and moderate precision,

leading to better balanced accuracies around 80-85%. The major caveat is that these numbers have only been obtained with minmax-scaled features. No scaling or normalization leads much more often to precision-recall pairs of 0% with balanced accuracies around 50%, indicating lots of type II errors.

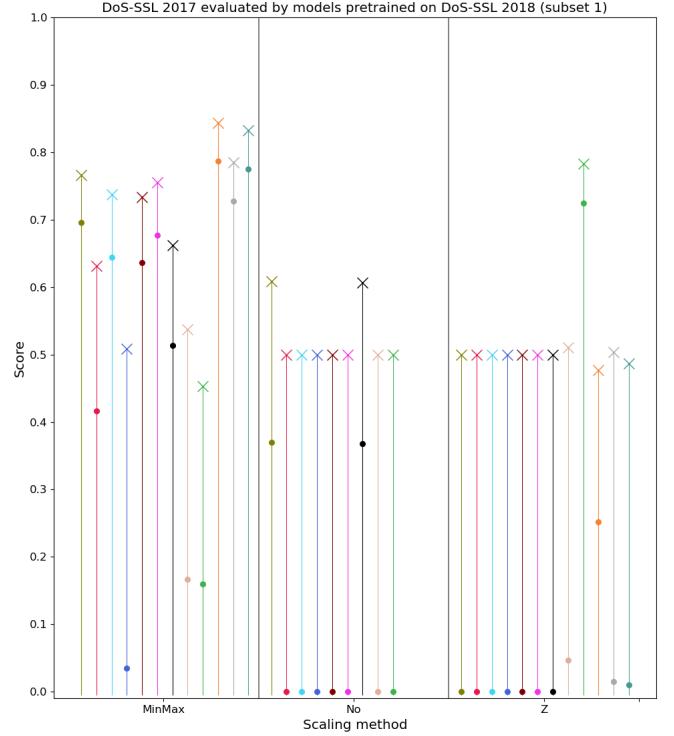


Figure 7: DoS-SSL 2017 evaluated by models trained on subset 1 of DoS-SSL 2018

The models of Dos subset 2 (heartleech) (fig. 8) of CSE-CIC-IDS2018 do not reach levels of classification performance that could be relied on in real intrusion detection systems. Most methods have recall-precision pairs at 0%, except when normalizing the features. Balanced accuracy reaches just under 80% in a singular case, with most methods performing around 60%. Spreads between precision and recall are either very large or zero, pushing F1-scores close to zero.

The overall result of fusing models trained on the DoS-SSL attack class of CSE-CIC-IDS2018 with the DoS-SSL data of CIC-IDS2017 is disappointing. The models trained on subset 1 of the 2018 data have some redeeming qualities, but the results are too inconsistent to be useful in a real sense.

The inversion of the previous scenario, feeding the 2 subsets containing DoS traffic from the 2018 dataset (1: layer-7 Dos, 2: heartleech) to the model trained on subset 1 of the 2017 dataset which bundled those attacks, does not achieve better results. The results are summarized in figures 9 and 10. When no scaling is applied to the features beforehand, all methods fail completely with F1 scores stably at 0%. Balanced accuracy still stands at 50%, meaning that the methods generated many false negatives. The RBF-kernel SVM and the logistic regression perform adequately in classifying the layer 7 DoS attacks, but only when minmax scaling was used prior to training (bal-

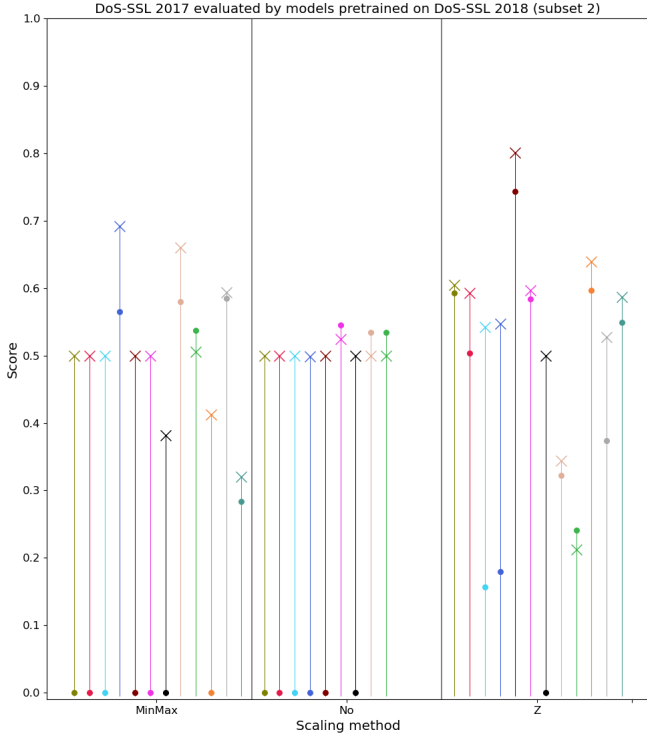


Figure 8: DoS-SSL 2017 evaluated by models trained on subset 2 of DoS-SSL 2018

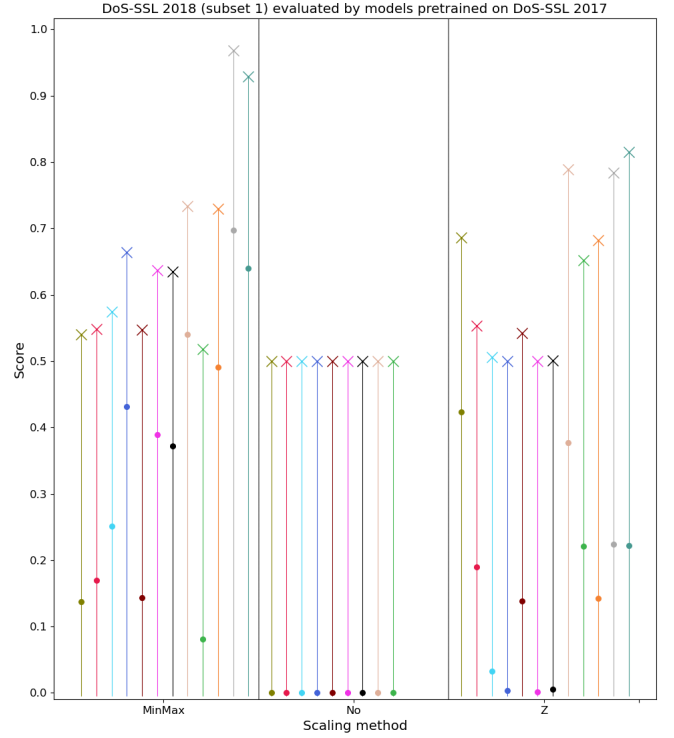


Figure 9: DoS-SSL subset 1 of 2018 evaluated by models trained on DoS-SSL 2017

anced accuracies, i.e. class separation above 95%). The other methods do not reach classification scores that are high enough to allow inclusion in the good performers. When using either type of scaling, the heartleech attack from the 2018 data is very well classified by the linear kernel SVM and the logistic regression trained on the 2017 data. The tree-based classifiers seem very brittle when tasked with classifying unseen but related data even though they boast the highest scores and broadest recognition when applied to the data sets (subsections 4.1 and 4.2). More work is needed to uncover if improved regularization can alleviate this problem.

5.3. Generalization of the Botnet Attack Class

The second class that was universally well-recognized in both datasets by the methods is botnet traffic. In the experimental layout for CIC-IDS2017 some nodes were infected with the ARES botnet to collect the exchanges between the command-and-control server and the bots. CSE-CIC-IDS2018 has patterns of infection by both the ARES and ZEUS botnets.

Generalization measured by exposure of the 2017 data to the 2018 models, yields unsatisfactory results. Figure 11 shows that some tree-based models manage class separability between 60 and 70% (minmax-scaling) with only slightly better results for the logistic regression and RBF-kernel SVM (65-80% balance accuracy). Despite traffic from the ARES botnet being present in both datasets, the pretrained models were not able to find a robust separation. Classification of the inverse situation (figure 12) shows much better results for models trained on normalized features. In this case the models could only derive a model of botnet behavior from the ARES botnet samples

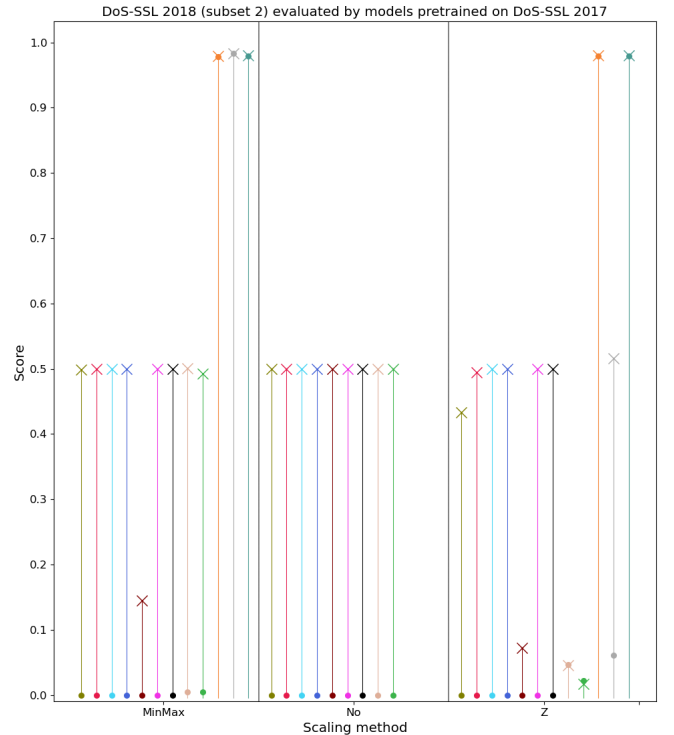


Figure 10: DoS-SSL subset 2 of 2018 evaluated by models trained on DoS-SSL 2017

in CIC-IDS2017. Several tree-based meta-estimators (bagging, adaboost and gradient-boosted tree classifiers) reach class-balanced accuracies around 75% (F1 $\sim$ 65%), with the best-overall method

the linear SVM which reaches 90% class-balanced accuracy (82% F1). Generalization on the whole is underwhelming, but some singular results stand out. More finely tuned models could perhaps reach even better scores. Nevertheless the discrepancies in model performance based on the preprocessing of the data is a concern as this is a weakness that was not present at all during classification within the data sets.

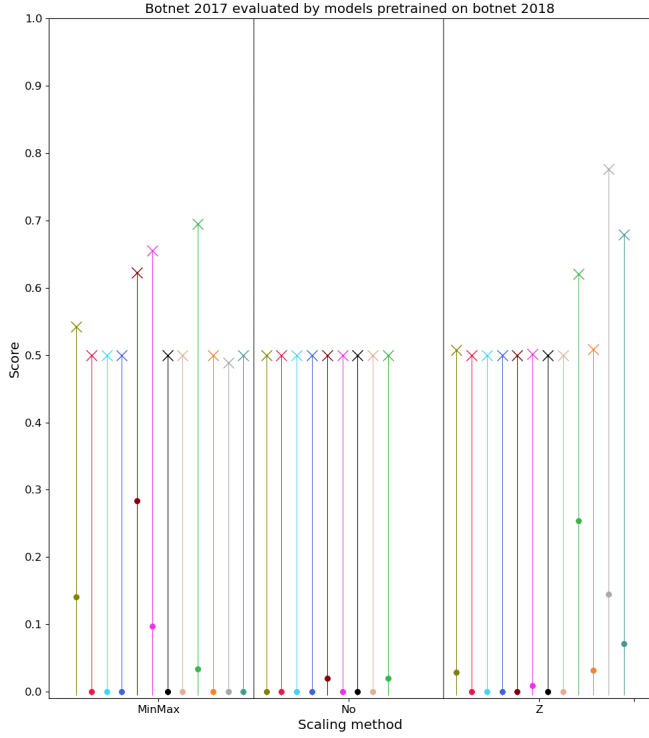


Figure 11: Botnet subset of 2017 evaluated by models trained on botnet 2018

6. Conclusions and Future Work

This article contains detailed experiment results on CIC-IDS2017 and CSE-CIC-IDS2018, two modern data sets geared towards the application of machine learning to network intrusion detection systems. The design and implementation have been laid out in section 3, focusing on the principles and application of solid machine learning engineering.

The baseline result section of this article (4) conveys the results of applying twelve supervised learning algorithms with optimized parameters to the data. Results were gathered for every individual subset, containing traffic from a specific attack class, as well as for a merged version (2017-only), containing all attack types.

In general, it can be stated that the tree-based classifiers performed best. Single decision trees are capable of recognizing DoS, DDoS and botnet traffic. Meta-estimators based on decision trees improved performance to the point where they are practically applicable for six of the seven attack classes. This was most true for the extratree- and XGBoost classifiers. Another improvement of meta-estimators over single decision trees is their ability to abstract over the choice of feature scaling.

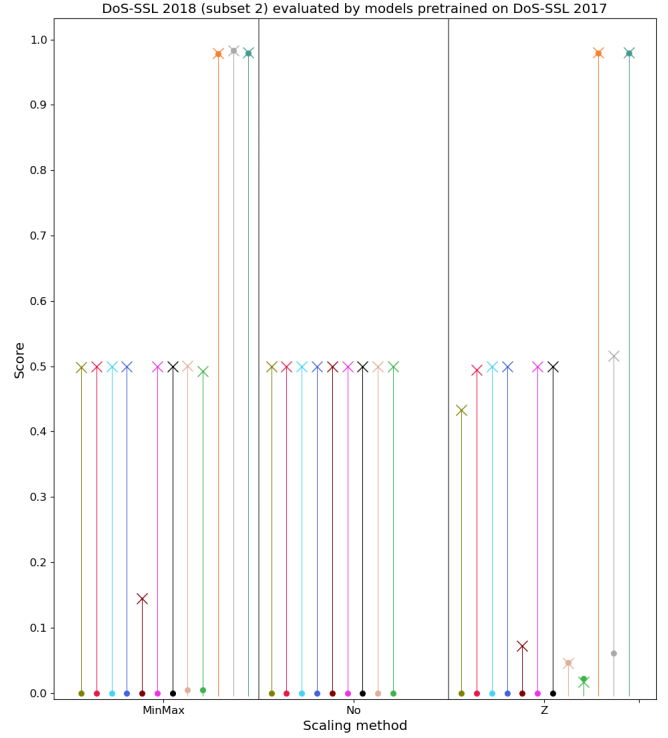


Figure 12: Botnet subset of 2018 evaluated by models trained on botnet 2017

In addition, performance on the merged data set was similarly high, without incurring heavy increases in execution time.

The elusive attack class to classify has been infiltration for both data sets. For the 2017-version the reason for this is likely the severe lack of positive training samples for this category, a natural consequence of the low network footprint of infiltration attacks. The 2018-version drastically ramps up the available infiltration samples, but for one of their days containing this type of attack it includes a Dropbox download in the malicious traffic. It is no stretch to hypothesize that a benign download from Dropbox looks no different from a malicious one at the network level.

Despite its simplicity k-nearest-neighbors is a potent classifier for five of the seven attack classes. It is held back by its steep increase in execution time for larger data sets, even though it generalizes as well as the tree-base meta-estimators. Opposite to this is the nearest centroid classifier, being nigh insensitive to dataset size, while applicable for the classification of port scan and DDoS traffic and for perfect detection of brute force and DoS traffic. The final algorithms, two support vector machines with different kernels and logistic regression are useful for recognition of port scan, DoS and DDoS traffic, provided the features are scaled (preferably normalized). However, these classifiers are not favored when pitted against the tree-based classifiers, because the attack classes on which they perform well are only a subset of the classes for which the tree-based perform equally well.

The second experiment tests the ability of the trained models to classify unseen samples of the same attack class from a different dataset. The DoS(-SSL) and botnet classes were

chosen, because they are very well recognized in both datasets across the different algorithms. The expectation was that the fusion of trained models on new, but very closely related data would keep the high classification scores.

This inter-dataset generalization testing of the 2017 data by 2018 models and vice versa, clearly shows that the generalization assumption is false. If no complete breakdown to 50% balanced accuracy is observed, precision and recall remain lackluster. In the handful of cases where an algorithm performs well, the results are not stable when it comes to the scaling choice. The progression of improved tree-based models that are supposed to improve generalization, only seem to do so within the test sets of the dataset on which they have been trained. This calls into question the usefulness of striving to combine or tweak methods that lead to marginal performance increments within one or more datasets while implying that the increase alone is sufficient proof that the method is better-suited at the task in general. In future work an analysis of top-published methods on the question of inter-dataset generalization will be carried out.

The final conclusion of this article is a critical remark on the usage of supervised learners in intrusion detection. Unless the results in this work are a consequence of methodological error, it would appear that supervised learning with classical methods succeeds very well within singular data sets, but fails to generalize performance even under optimal testing conditions. CIC-IDS2017 and CSE-CIC-IDS2018 differ only very slightly from a methodological perspective. Further research is necessary to confirm that these losses in classification performance also exist for the other attack types. Beyond that first next step, an investigation into whether the root cause is a data problem (e.g. insufficient variance in attack or baseline traffic, different patterns in baseline traffic that should be harmonized before classification, in- or over-significance of features in the CIC-collection, etc.) or a method problem (e.g. the patterns are too complex to be extracted by the tested algorithms, the mechanisms for regularization should be improved, etc.).

References

- Attak, H., Combalia, M., Gardikis, G., Gastón, B., Jacquin, L., Litke, A., Papadakis, N., Papadopoulos, D., & Pastor, A. (). Application of distributed computing and machine learning technologies to cybersecurity. *Space*, 2, 12CAT.
- Axelsson, S. (2000). *Intrusion detection systems: A survey and taxonomy*. Technical Report Technical report.
- Brown, C., Cowperthwaite, A., Hijazi, A., & Somayaji, A. (2009). Analysis of the 1999 darpa/lincoln laboratory ids evaluation data with netadht. In *Computational Intelligence for Security and Defense Applications, 2009. CISDA 2009. IEEE Symposium on* (pp. 1–7). IEEE.
- Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18, 1153–1176.
- Chen, T., & Guestrin, C. (2016). Xgboost: A scalable tree boosting system. *CoRR, abs/1603.02754*. URL: <http://arxiv.org/abs/1603.02754>. arXiv:1603.02754.
- Cloudflare-blog (). Inside the infamous mirai iot botnet: A retrospective analysis. URL: <https://blog.cloudflare.com/inside-mirai-the-infamous-iot-botnet-a-retrospective-analysis/>.
- Creech, G., & Hu, J. (2013). Generation of a new ids test dataset: Time to retire the kdd collection. In *2013 IEEE Wireless Communications and Networking Conference (WCNC)* (pp. 4487–4492). IEEE.
- Das, S., Mahfouz, A. M., Venugopal, D., & Shiva, S. (2019). Ddos intrusion detection through machine learning ensemble. In *2019 IEEE 19th International Conference on Software Quality, Reliability and Security Companion (QRS-C)* (pp. 471–477).
- Das, S., Venugopal, D., & Shiva, S. (2020). A holistic approach for detecting ddos attacks by using ensemble unsupervised machine learning. In K. Arai, S. Kapoor, & R. Bhatia (Eds.), *Advances in Information and Communication* (pp. 721–738). Cham: Springer International Publishing.
- Denning, D., & Neumann, P. G. (1985). *Requirements and model for IDES-a real-time intrusion-detection expert system*. SRI International.
- Folino, G., & Sabatino, P. (2016). Ensemble based collaborative and distributed intrusion detection systems: A survey. *Journal of Network and Computer Applications*, 66, 1 – 16. URL: "<http://www.sciencedirect.com/science/article/pii/S1084804516300248>". doi:<https://doi.org/10.1016/j.jnca.2016.03.011>.
- Gao, X., Shan, C., Hu, C., Niu, Z., & Liu, Z. (2019). An adaptive ensemble machine learning model for intrusion detection. *IEEE Access*, 7, 82512–82521.
- Gates, C., & Taylor, C. (2007). Challenging the anomaly detection paradigm: A provocative discussion. In *Proceedings of the 2006 Workshop on New Security Paradigms NSPW '06* (pp. 21–29). New York, NY, USA: ACM. URL: <http://doi.acm.org/10.1145/1278940.1278945>. doi:10.1145/1278940.1278945.
- Hao, S., Long, J., & Yang, Y. (2019). BI-ids: Detecting web attacks using bi-lstm model based on deep learning. In *International Conference on Security and Privacy in New Computing Environments* (pp. 551–563). Springer.
- Hellemons, L., Hendriks, L., Hofstede, R., Sperotto, A., Sadre, R., & Pras, A. (2012). Sshcure: a flow-based ssh intrusion detection system. In *IFIP International Conference on Autonomous Infrastructure, Management and Security* (pp. 86–97). Springer.
- Hodo, E., Bellekens, X., Hamilton, A., Tachtatzis, C., & Atkinson, R. (2017). Shallow and deep networks intrusion detection system: A taxonomy and survey. *arXiv preprint arXiv:1701.02145*.
- Jazi, H. H., Gonzalez, H., Stakhanova, N., & Ghorbani, A. A. (2017). Detecting http-based application layer dos attacks on web servers in the presence of sampling. *Computer Networks*, 121, 25–36.
- Liu, Z., Zhu, Y., Yan, X., Wang, L., Jiang, Z., Luo, J. et al. (2020). Deep learning approach for ids. In *Fourth International Congress on Information and Communication Technology* (pp. 471–479). Springer.
- Małowidzki, M., Berezinski, P., & Mazur, M. (2015). Network intrusion detection: Half a kingdom for a good dataset. In *Proceedings of NATO STO SAS-139 Workshop, Portugal*.
- Marir, N., Wang, H., Feng, G., Li, B., & Jia, M. (2018). Distributed abnormal behavior detection approach based on deep belief network and ensemble svm using spark. *IEEE Access*, 6, 59657–59671.
- McHugh, J. (2000). The 1998 lincoln laboratory ids evaluation. In H. Debar, L. Mé, & S. F. Wu (Eds.), *Recent Advances in Intrusion Detection* (pp. 145–161). Berlin, Heidelberg: Springer Berlin Heidelberg.
- McKinney, W. (2010). Data structures for statistical computing in python. In S. van der Walt, & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 51 – 56).
- Mirsky, Y., Doitshman, T., Elovici, Y., & Shabtai, A. (2018). Kitsune: an ensemble of autoencoders for online network intrusion detection. *arXiv preprint arXiv:1802.09089*.
- Moustafa, N., Turnbull, B., & Choo, K.-K. R. (2018). An ensemble intrusion detection technique based on proposed statistical flow features for protecting network traffic of internet of things. *IEEE Internet of Things Journal*, 6, 4815–4830.
- Papamartzivanos, D., Mármol, F. G., & Kambourakis, G. (2019). Introducing deep learning self-adaptive misuse network intrusion detection systems. *IEEE Access*, 7, 13546–13560.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Probst T., K. M., Alata E., & V., N. (2015). Automated evaluation of network intrusion detection systems in iaas clouds. In *2015 11th European Dependable Computing Conference (EDCC)* (pp. 49–60). doi:10.1109/

EDCC.2015.10.

- Robertson, W., Vigna, G., Kruegel, C., Kemmerer, R. A. et al. (2006). Using generalization and characterization techniques in the anomaly-based detection of web attacks. In *NDSS*.
- Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *ICISSP* (pp. 108–116).
- Shiravi, A., Shiravi, H., Tavallaee, M., & Ghorbani, A. A. (2012). Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *computers & security*, 31, 357–374.
- Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE Symposium on Security and Privacy* (pp. 305–316). doi:10.1109/SP.2010.25.
- Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the kdd cup 99 data set. In *Computational Intelligence for Security and Defense Applications, 2009. CISDA 2009. IEEE Symposium on* (pp. 1–6). IEEE.
- Vaca, F. D., & Niyaz, Q. (2018). An ensemble learning based wi-fi network intrusion detection system (wnids). In *2018 IEEE 17th International Symposium on Network Computing and Applications (NCA)* (pp. 1–5).
- Viegas, E. K., Santin, A. O., & Oliveira, L. S. (2017). Toward a reliable anomaly-based intrusion detection in real-world environments. *Computer Networks*, 127, 200 – 216. URL: <http://www.sciencedirect.com/science/article/pii/S1389128617303225>. doi:<https://doi.org/10.1016/j.comnet.2017.08.013>.
- Wang, S., Xia, C., & Wang, T. (2019). A novel intrusion detector based on deep learning hybrid methods. In *2019 IEEE 5th Intl Conference on Big Data Security on Cloud (BigDataSecurity), IEEE Intl Conference on High Performance and Smart Computing (HPSC) and IEEE Intl Conference on Intelligent Data and Security (IDS)* (pp. 300–305). IEEE.
- Wu, S. X., & Banzhaf, W. (2010). The use of computational intelligence in intrusion detection systems: A review. *Applied soft computing*, 10, 1–35.
- Yin, C., Zhu, Y., Fei, J., & He, X. (2017). A deep learning approach for intrusion detection using recurrent neural networks. *Ieee Access*, 5, 21954–21961.
- Zinkevich, M. (). Rules of machine learning: Best practices for ml engineering. URL: <https://developers.google.com/machine-learning/guides/rules-of-ml/>.

Appendix

6.1. CIC-IDS2017 & CSE-CIC-IDS2018 Full Tabular Results and Graphics

Tables that contain all results of the analyses of both data sets and their visualizations can be found at <https://gitlab.ilabt.imec.be/lpdhooge/CIC-IDS2017-2018-ml-graphics>.

6.2. Model Generalization Experiment Source Code

The source code as well as information on the prerequisites and execution of the inter-dataset generalization experiment is publicly available at <https://gitlab.ilabt.imec.be/lpdhooge/model-unseen-testing-cicids>.